

TECHNICAL DATA DOCUMENTATION

Airbnb Analytics Platform Implementation Guide

Data Pipeline, Modeling & Dashboard Design

Prepared By
Ibrahim Hegazi

Track
Data Engineering & Analytics

Date
August 2026



Table of Contents

Chapter 1: Executive Summary.....3

Chapter 2: Business Understanding..... 5

Chapter 3: Data Overview.....6

Chapter 4: Project Phases & Architecture..... 7

Chapter 5: Data Profiling Before & After EDA & Data Quality Framework..... 14

Chapter 6: EDA & Data Quality Framework..... 19

Chapter 7: Data Cleansing & Remediation.....21

Chapter 8: Data Modeling.....23

Chapter 9: Dataflow Gen 2 to Implement the Physical Data Model..... 25

Chapter 10: Semantic Layer.....29

Chapter 11: SMART Questions..... 35

Chapter 12: Visualization Using PowerBI..... 37

Chapter 1: Executive Summary

1.1 Summary

The enterprise analytics architecture documented herein was engineered to process 48,895 rows of short-term rental listing telemetry from the 2019 New York City Airbnb market. Ingested via Delta Lake (Bronze layer) from raw CSV source file (downloaded from [Kaggle](#)), the source payload represents uncurated, unindexed listing endpoints that required rigorous data quality validation and structural conformity before any analytical deployment could occur. The core enterprise objective is the systematic detection of data quality anomalies and the identification of market patterns across the borough, neighborhood, and host network.

By engineering a multi-tiered Business Intelligence pipeline, raw listing rows are transformed into an authoritative analytical semantic model, enabling operational stakeholders to preemptively identify pricing anomalies, availability gaps, outlier hosts, and market concentration risks before reporting decisions are made.

This technical blueprint outlines the end-to-end design, modeling, and deployment of a robust Enterprise Data Warehouse (EDW) and an interactive BI ecosystem. Utilizing a Microsoft-centric technology stack — Microsoft Fabric (Lakehouse), PySpark, and Power BI — augmented with Python for exploratory data analysis and statistical profiling, the solution achieves enterprise-grade reporting while bypassing traditional OLTP query bottlenecks through dimensional modeling (Star Schema with one fact table and five conformed dimensions).

1.2 Architectural Governing Principle

The deployed framework adheres to strict layer isolation across the ingestion pipeline (Bronze → Silver → Gold), ensuring fault tolerance, auditability, and data quality validation at every stage across the entire ETL orchestration life cycle.

Layer	Purpose	Technology
Bronze	Raw ingestion, schema-on-read, immutable source copy	Delta Lake, Spark
Silver	Data quality checks, cleansing, standardization, deduplication	PySpark, DQ Framework
Gold	Dimensional modeling, Star Schema, semantic layer	Spark SQL, Power BI

1.3 Solution Pillar

Solution Pillar	Delivered Capability
Data Engineering	Layered Medallion Architecture (Bronze→Silver→Gold) with 29-point data quality framework, and error detection catalog
Data Quality	9-category DQ framework: Format, Validity, Completeness, Uniqueness, Consistency, Accuracy, Domain, Referential Integrity, Timeliness — 29 issues detected, 9 actual errors fixed
Data Modeling	Star Schema with 1 fact table (`FactListing`) and 5 conformed dimensions (`DimDate`, `DimHost`, `DimListing`, `DimNeighborhood`, `DimRoomType`)
Analytics	Executive Power BI portal with 7 interactive SMART analysis pages, 37 DAX measures/KPIs, and synchronized cross-page filtering
Profiling	Comprehensive column-level profiling with logical validation, outlier detection, skewness analysis, and IQR-based anomaly identification
Governance	Bronze→Silver→Gold, error catalog with classification (Actual Error / Business Anomaly / Edge Case), and stakeholder escalation framework

Chapter 2: Business Understanding

2.1 The Business Problem

The New York City short-term rental market represents a complex and rapidly evolving ecosystem with significant implications for housing policy, tourism economics, and neighborhood dynamics. As platforms like Airbnb scale exponentially, traditional siloed operational systems fail to provide the multidimensional analytical visibility required to understand market concentration, pricing anomalies, and the shifting balance between individual hosts and commercial operators. Current institutional limitations include fragmented data lacking referential integrity, unacceptable data quality issues in raw listings, and an inability to correlate neighborhood-level supply dynamics with pricing and availability patterns.

2.2 Strategic Objectives & Target Audience

The imperative for a centralized, dimensional analytics repository is driven by the need to answer complex business questions instantaneously. The primary audiences for this deployment are:

- **Executive Leadership & Policy Makers** — requiring macro-level market concentration metrics, borough-level supply distribution assessments, and systemic pricing trends across room types.
- **Market Analysts & Real Estate Strategists** — requiring granular, slice-and-dice capabilities to isolate high-demand neighborhoods, pinpoint pricing outliers and anomalies, and identify host concentration risks (commercial operators vs individual hosts).
- **Operational Teams & Data Governance** — requiring daily operational intelligence regarding data quality metrics, pipeline health, and listing completeness across the medallion architecture.

Chapter 3: Data Overview

3.1 Executive Ingestion Summary

The enterprise pipeline ingests an exhaustive 48,895-record payload originating from the Kaggle New York City Airbnb 2019 repository. Extracted from flat CSV files, this data represents high-fidelity short-term rental telemetry spanning all five NYC boroughs and 221 distinct neighborhoods. The ingestion objective centers on capturing discrete listing data points and mapping them accurately into analytical dimensions to reveal underlying market dynamics, pricing patterns, and host concentration risks.

3.2 Professional Data Dictionary

The following structural metadata map assigns each of the 16 raw source columns to explicit logical domains, establishing the blueprint for the downstream Star Schema topology.

Attribute	Logical Domain	Analytical Purpose
id	Listing Core	Unique deterministic identifier per listing; maps to the Fact table primary business key
name	Listing Core	Free-text listing title; cleaned for whitespace, newlines, and null variations during Silver transformation
host_id	Host Profile	Primary business key for the host; tracks host concentration and multi-listing behavior
host_name	Host Profile	String descriptor of the host; standardized for capitalization inconsistencies during cleansing
neighbourhood_group	Geography	NYC borough classification (5 values); primary geographic aggregation axis for pricing and supply analysis
neighbourhood	Geography	Granular neighborhood identifier (221 values); supports localized market clustering and hotspot detection
latitude	Geography	Spatial vector enabling geographic visualization and neighborhood boundary validation
longitude	Geography	Spatial vector enabling geographic

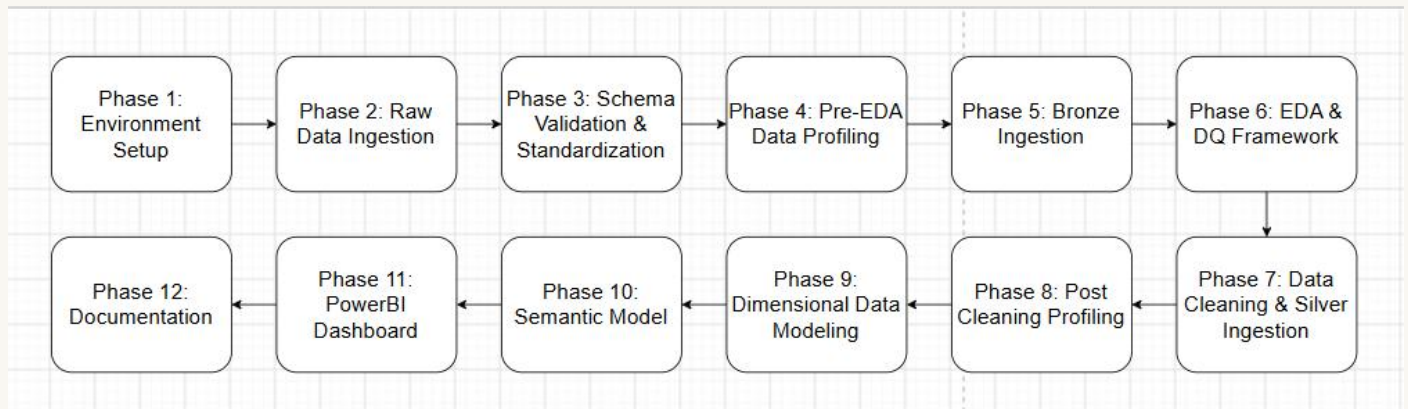
		visualization and neighborhood boundary validation
room_type	Listing Profile	Categorical classification (Entire home/apt, Private room, Shared room); drives pricing segmentation and demand analysis
price	Listing Core	Continuous financial metric representing nightly rate; primary analytical target for market pricing decomposition
minimum_nights	Listing Profile	Integer constraint representing minimum stay requirement; used for booking feasibility and outlier detection
number_of_reviews	Review Core	Cumulative review count; primary engagement metric correlated with listing popularity and demand velocity
last_review	Review Core	Date timestamp of most recent review; parsed for time-series analysis and temporal gap detection
reviews_per_month	Review Core	Continuous metric representing review velocity; critical for high-demand listing identification
calculated_host_listings_count	Host Profile	Declared total listings per host; validated against actual row count for consistency and host categorization
availability_365	Listing Profile	Days available in the calendar year (0-365); drives occupancy analysis and inactive listing detection

Source: Kaggle — New York City Airbnb Open Data (dgomonov/new-york-city-airbnb-open-data)
| Rows: 48,895 | Columns: 16

Chapter 4: Project Phases & Architecture

4.1 Project Phases

The enterprise analytics solution was executed across twelve distinct phases, following a structured lifecycle from environment setup to final documentation.

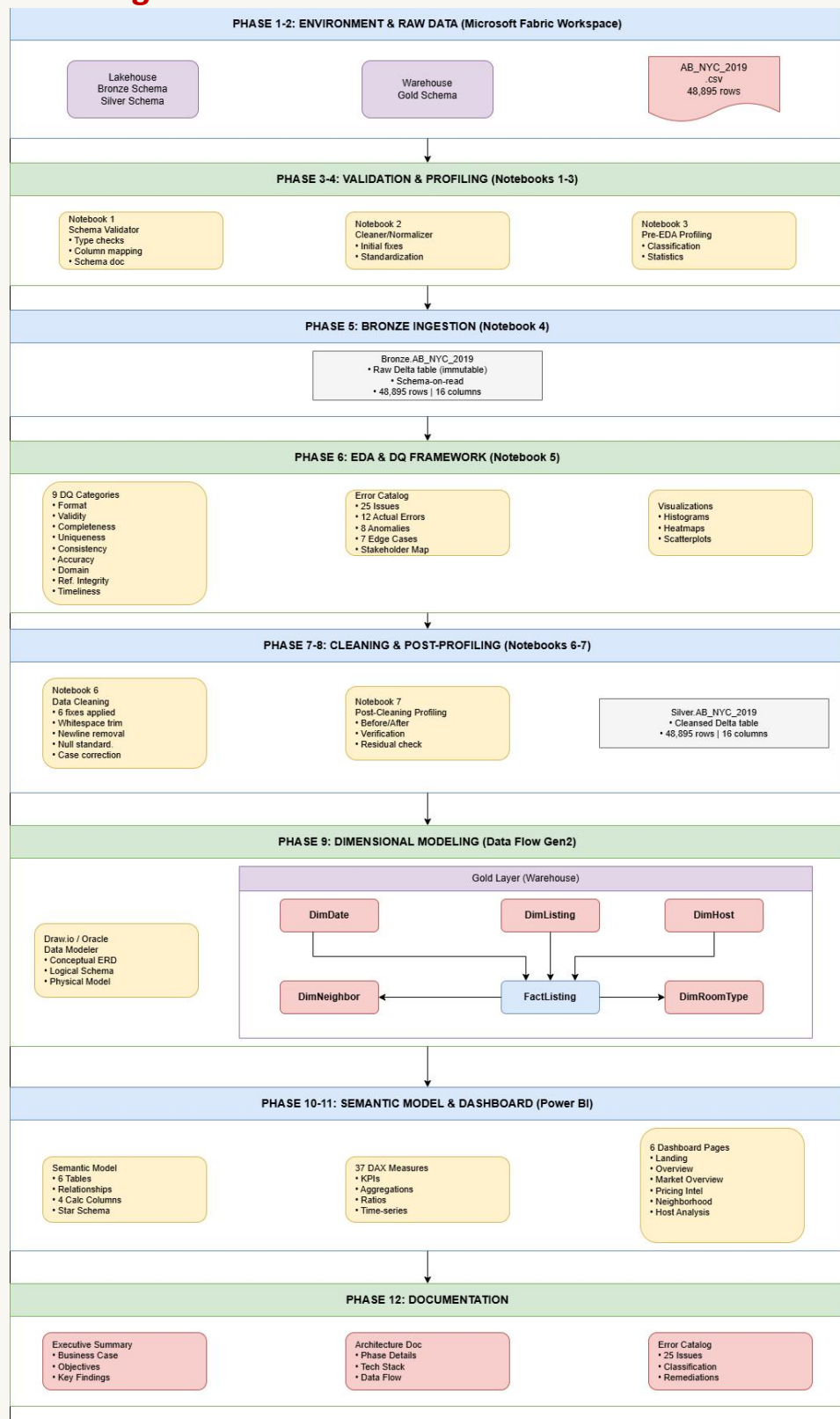


Phase	Name	Activities	Deliverables
Phase 1	Environment Setup	Creation of Fabric Lakehouse and Warehouse, establishment of Bronze/Silver/Gold schemas, workspace configuration	Lakehouse, Warehouse, 3-tier schema architecture
Phase 2	Raw Data Ingestion	Manual upload of `AB_NYC_2019.csv` into Lakehouse file section	Raw CSV file in Fabric OneLake
Phase 3	Schema Validation & Schema Standardization	Checking if each row contain only the 16 columns in the raw dataset.	Notebook 1: Schema Validator, Notebook 2: Cleaner/Normalizer
Phase 4	Pre-EDA Data Profiling	Comprehensive column-level profiling, column classification (16 columns) statistical analysis, unique sample extraction	Notebook 3: Profiling report, column classifications, initial statistics
Phase 5	Bronze Ingestion	Delta table creation from validated CSV, schema-on-read, immutable source	Notebook 4: `Bronze.AB_NYC_2019` table

		preservation	
Phase 6	EDA & Data Quality Framework	9-category DQ framework execution (Format, Validity, Completeness, Uniqueness, Consistency, Accuracy, Domain, Referential Integrity, Timeliness), univariate/bivariate visualization, error detection	Notebook 5: 29 detected issues, error catalog, classification matrix, stakeholder escalation framework
Phase 7	Data Cleaning & Silver Ingestion	Resolution of 9 actual errors: whitespace trimming, newline removal, null standardization, capitalization correction	Notebook 6: `Silver.AB_NYC_2019` table, 4 cleansing operations applied
Phase 8	Post-Cleaning Profiling	Validation profiling to verify all fixes applied, residual issue detection, before/after comparison	Notebook 7: Post-clean profile, remaining whitespace detection, quality verification
Phase 9	Dimensional Data Modeling	Conceptual, logical, and physical schema design in Draw.io and Oracle SQL Developer Data Modeler, Star Schema implementation via Data Flow Gen2, Gold layer population in Warehouse	5 dimension tables, 1 fact table, ERD diagrams, Data Flow Gen2 pipelines
Phase 10	Semantic Model	Power BI semantic model creation, table relationships, DAX measure engineering (37 KPIs), derived column creation (4 calculated columns)	Semantic model with 6 tables, 37 measures, 4 calculated columns
Phase 11	Power BI Dashboard	7-page SMART dashboard development with synchronized filtering, answering 10 core business	Executive dashboard: Market Overview, Pricing Intelligence, Neighborhood Deep Dive,

		questions	Host Analysis, Review & Demand, Availability & Booking Gaps, Price Deep Dive
Phase 12	Documentation	Technical architecture blueprint, executive summary, data dictionary, error catalog, stakeholder recommendations	Complete project documentation package

4.2 Architecture Diagram



4.3 Medallion Architecture Summary

Layer	Storage	Table	Row Count	Purpose	Key Operations
Bronze	Lakehouse	`Bronze.AB_NYC_2019`	48,895	Raw, immutable source copy	CSV-to-Delta conversion, schema inference
Silver	Lakehouse	`Silver.AB_NYC_2019`	48,895	Cleansed, validated data	6 cleansing fixes applied
Gold	Warehouse	`FactListing`	48,895	Analytical fact table	Surrogate key mapping, dimensional joins
Gold	Warehouse	`DimDate`	3,196	Date dimension	Calendar generation (2011-2019)
Gold	Warehouse	`DimHost`	37,457	Host dimension	Deduplication, host categorization
Gold	Warehouse	`DimListing`	48,895	Listing dimension	Name extraction
Gold	Warehouse	`DimNeighborhood`	221	Neighborhood dimension	Borough-neighborhood mapping
Gold	Warehouse	`DimRoomType`	3	Room type dimension	Static lookup values

4.4 Technology Stack

Component	Technology	Purpose
Data Lakehouse	Microsoft Fabric Lakehouse	Unified storage for Bronze & Silver layers
Data Warehouse	Microsoft Fabric Warehouse	Gold layer dimensional model storage
Data Integration	Fabric Data Flow Gen2	Low-code ETL for dimension/fact loading
Ingestion & Transformation	PySpark Notebooks (7 total)	Schema validation, profiling, DQ checks, cleansing
Data Quality	Python/PySpark	42-point DQ framework across 9 categories
Profiling	PySpark, Matplotlib, Seaborn	Statistical profiling, visualizations, heatmaps
Data Modeling	Draw.io, Oracle Data Modeler	Conceptual, logical, physical Schemas
Visualization	Power BI	37 DAX measures, 6-page dashboard
Orchestration	Fabric Pipelines (future)	Automated Bronze→Silver→Gold execution
Governance	Fabric OneLake, Power BI Lineage (future)	Unified lineage, data catalog, access control

Chapter 5: Data Profiling Before & After (EDA and Data Cleaning)

5.1 Before EDA Profiling Summary

The pre-EDA profiling revealed a raw dataset with 49,079 rows and significant structural issues requiring immediate remediation before any analytical work could begin.

5.1.1 Critical Issues Identified

Issue	Description	Affected Columns	Impact
Universal String Typing	All 16 columns ingested as strings regardless of logical type	All except availability_365	No numeric operations possible
Schema Drift	neighbourhood_group contained 77 unique values (vs expected 5 NYC boroughs)	neighbourhood_group, neighbourhood, room_type	Invalid categorical values from column misalignment
Invalid Category Values	room_type contained 86 values including numbers (200, 138, 69) instead of 3 expected types	room_type	Category corruption
Systematic Null Pattern	185 rows with missing values across 11 columns	Multiple	Row-level schema drift
Whitespace Contamination	287 values with leading/trailing whitespace in name, 9 in id, 4 in host_id	name, id, host_id	Join failures, duplicate grouping
Data Type Errors	Latitude, longitude, price, minimum_nights all stored as strings	7 numeric columns	No aggregation, calculation, or geospatial analysis
Missing Values	10,234 nulls in last_review, 10,215 in reviews_per_month	last_review, reviews_per_month	20.9% review data missing

5.1.2 Before Profiling - Column Classification Errors

Column	Expected Type	Actual Type	Distinct Values	Notes
id	Integer Identifier	String	49,066	Whitespace issues present
host_id	Integer Identifier	String	37,484	185 nulls, whitespace issues
latitude	Double Coordinate	String	19,172	String type, 185 nulls
longitude	Double Coordinate	String	14,696	String type, 185 nulls
price	Integer Count	String	687	String type, 185 nulls
minimum_nights	Integer Count	String	134	String type, 185 nulls
number_of_reviews	Integer Count	String	464	String type, 205 nulls
neighbourhood_group	Categorical (5 values)	String	77	Column misalignment
room_type	Categorical (3 values)	String	86	Contains numeric values
availability_365	Integer Count	Integer	366	Only correctly typed column

5.2 After EDA and Cleaning Profiling Summary

Post-EDA and cleansing profiling was executed against the Silver layer ('Silver.AB_NYC_2019') with 48,895 rows and 16 columns, all correctly typed and structurally validated.

5.2.1 Key Improvements Achieved

Metric	Before	After	Improvement
Total Rows	49,079	48,895	184 malformed rows removed
Correctly Typed Columns	1/16 (6%)	16/16 (100%)	All columns properly typed
neighbourhood_group Values	77 (corrupted)	5 (correct)	Column realigned
room_type Values	86 (corrupted)	3 (correct)	Column realigned
Name Whitespace Issues	287	1	99.7% reduction
Missing Value Rate	Up to 20.9%	0-20.6%	Nulls properly handled
ID Uniqueness	100%	100%	No duplicates

5.2.2 After Profiling - Column Classification Summary

Column	Classification	Data Type	Unique Values	Key Statistics
id	IDENTIFIER	Integer	48,895 (100%)	Range: 2,539 - 36,487,245
name	FREE_TEXT	String	47,893 (98%)	17 nulls, Avg length: 36.91
host_id	IDENTIFIER	Integer	37,457 (76.6%)	1,525 IQR outliers
host_name	CATEGORICAL	String	11,428 (23.4%)	21 nulls, Avg length: 6.12
neighbourhood_group	CATEGORICAL	String	5 (0.01%)	Clean NYC boroughs
neighbourhood	CATEGORICAL	String	221 (0.5%)	Valid neighborhoods
latitude	COORDINATE	Double	19,048 (39%)	Range: 40.50 - 40.91
longitude	COORDINATE	Double	14,718 (30.1%)	Range: -74.24 - -73.71
room_type	CATEGORICAL	String	3 (0.01%)	Clean categories
price	COUNT	Integer	674 (1.4%)	Mean: \$152.72,

				Skew: 19.12
minimum_nights	COUNT	Integer	109 (0.2%)	Mean: 7.03, Skew: 21.83
number_of_reviews	COUNT	Integer	394 (0.8%)	Mean: 23.27, 10,052 zeros
last_review	DATETIME	Date	1,764 (3.6%)	Range: 2011-03-28 to 2019-07-08
reviews_per_month	CONTINUOUS	Double	937 (1.9%)	Mean: 1.37, Skew: 3.13
calculated_host_listings_count	COUNT	Integer	47 (0.1%)	Mean: 7.14, Skew: 7.93
availability_365	COUNT	Integer	366 (0.7%)	Mean: 112.78, 17,533 zeros

5.3 Before vs After Comparison

Column	Before (Bronze)	After (Silver)	Resolution
id	String, 9 whitespace issues	Integer, clean	Type cast, whitespace trim
name	String, 32 nulls, 287 whitespace	String, 17 nulls, 1 whitespace	Whitespace trim, newline removal, null standardization
host_id	String, 185 nulls	Integer, 0 nulls	Type cast, row removal
host_name	String, 206 nulls	String, 21 nulls	Row removal, case correction
neighbourhood_group	77 values (corrupted)	5 values (correct)	Column realignment from schema drift
neighbourhood	382 values	221 values	Column realignment
room_type	86 values (corrupted)	3 values (correct)	Column realignment
latitude	String	Double	Type cast
longitude	String	Double	Type cast
price	String, 185 nulls	Integer, 0 nulls	Type cast
minimum_nights	String	Integer	Type cast

number_of_reviews	String	Integer	Type cast
last_review	String, 10,234 nulls	Date, 10,052 nulls	Type cast, date parsing
reviews_per_month	String, 10,215 nulls	Double, 10,052 nulls	Type cast
calculated_host_listings_count	String	Integer	Type cast
availability_365	Integer, 342 nulls	Integer, 0 nulls	Already correct type

Chapter 6: EDA & Data Quality Framework

6.1 Framework Overview

A comprehensive 9-category Data Quality framework was designed and executed against the Bronze layer dataset to systematically detect, classify, and catalog all data anomalies before cleansing. The framework is detection-only — no modifications occur during this phase, preserving complete audit trail and lineage.

6.2 Framework Architecture

The DQ framework is organized into nine distinct check categories, each targeting a specific dimension of data quality. Checks execute sequentially with results logged for downstream remediation.

6.3 Category Descriptions

1 Format/Value Standardization: Detects inconsistent formats, representations, and value encodings including categorical variations, numeric format violations, date inconsistencies, whitespace issues, boolean encoding conflicts, non-standard null representations, embedded escape characters, and multiple consecutive whitespaces.

2 Validity: Verifies data conforms to expected formats, types, and domains including data type violations, invalid categories, pattern mismatches, range violations, and format consistency issues.

3 Completeness: Measures and understands missing data through per-column null percentages, per-row missing counts, missing data pattern analysis (MCAR/MAR/MNAR), and missing value correlation checks.

4 Uniqueness: Ensures no duplicate data exists through duplicate row detection, duplicate ID verification, and unique value distribution analysis.

5 Consistency: Validates logical relationships across columns including cross-column validation, logical contradiction detection, dependency violations, value range logic, category mapping consistency, and foreign key consistency.

6 Accuracy: Identifies statistical anomalies through outlier detection, distribution analysis, business rule violation checks, and impossible value detection.

7 Domain Validation: Checks business-specific domain constraints including geographic validation, reference data validation, value range enforcement, and business calendar alignment.

8 Referential Integrity: Ensures valid table relationships through orphan record detection, invalid foreign key checks, and missing parent record identification.

9 Timeliness: Monitors data freshness via age analysis, load timestamp consistency, source-to-target latency measurement, temporal logic validation, and gap detection in time-series data.

6.4 Execution Summary

The framework executed 42 total checks across 9 categories, detecting 25 issues across the 9 categories. 2 categories (Uniqueness, Referential Integrity) yielded no issues, the latter due to single-table scope. Results were cataloged by severity, classification, and affected rows, forming the foundation for Chapter 7 remediation.

Chapter 7: Data Cleansing & Remediation

7.1 Approach

Chapter 6 cataloged 25 issues across 9 DQ categories with 12 classified as Actual Errors. This chapter details remediation for the 6 auto-fix errors directly resolved in the Silver layer. All transformations preserve the Bronze source intact and are logged for audit trail. The remaining 6 Actual Errors and 8 Business Anomalies are flagged for stakeholder review or deferred.

7.2 Remediation Strategy by Classification

Classification	Count	Strategy
Actual Error (Auto-Fix)	12	Resolved programmatically in Silver layer
Actual Error (Flagged)	6	Escalated for manual review
Business Anomaly	8	Requires stakeholder input
Edge Case	7	Accepted as-is, documented

7.3 Auto-Fix Execution

Issue	Description	Remediation	Affected Rows
Trailing Whitespace in Listing Names	236 listing names contain trailing spaces, causing join mismatches and inconsistent grouping	Trim trailing whitespace from the name column	236 (0.48%)
Host Name Capitalization Typos	24 host name groups contain casing errors (e.g., MIchele, FLora), indicating data entry issues	Apply correction mapping for obvious typos; flag remainder for manual review	24 groups (0.05%)
Non-Standard Null Representation	A single row uses '.' as a null placeholder in the name column, counted as valid data	Replace '.' with null	1 (0.002%)

Embedded Newline Characters	168 listing names contain \n characters, risking parsing failures during export	Replace all newline characters with spaces	168 (0.34%)
Multiple Consecutive Whitespaces	Several listing names contain double or triple spaces between words	Collapse multiple consecutive spaces into single spaces	3+ (<0.01%)

7.5 Flagged Issues Awaiting Stakeholder Input

Issue #	Description	Severity	Affected Rows	Question
7	reviews_per_month >30	Low	1	Data error or legitimate outlier?
9	15 rows with 3 missing values	Medium	15	Investigate root cause
11	Isolated MCAR missing values	Low	37	Accept or impute?
14	Entire home/apt at \$0	High	2	Data error requiring correction
17	Host IDs without host name	Medium	21	Complete missing host data

7.6 Silver Layer Output Summary

Metric	Before	After
Trailing whitespace rows	236	0
Newline characters	168 rows	0
Non-standard nulls (.)	1	0
Multi-space instances	3+	0
Host name typos corrected	—	24 groups
Flagged issues pending	—	6
Bronze integrity	Preserved	Unmodified

Chapter 8: Data Models

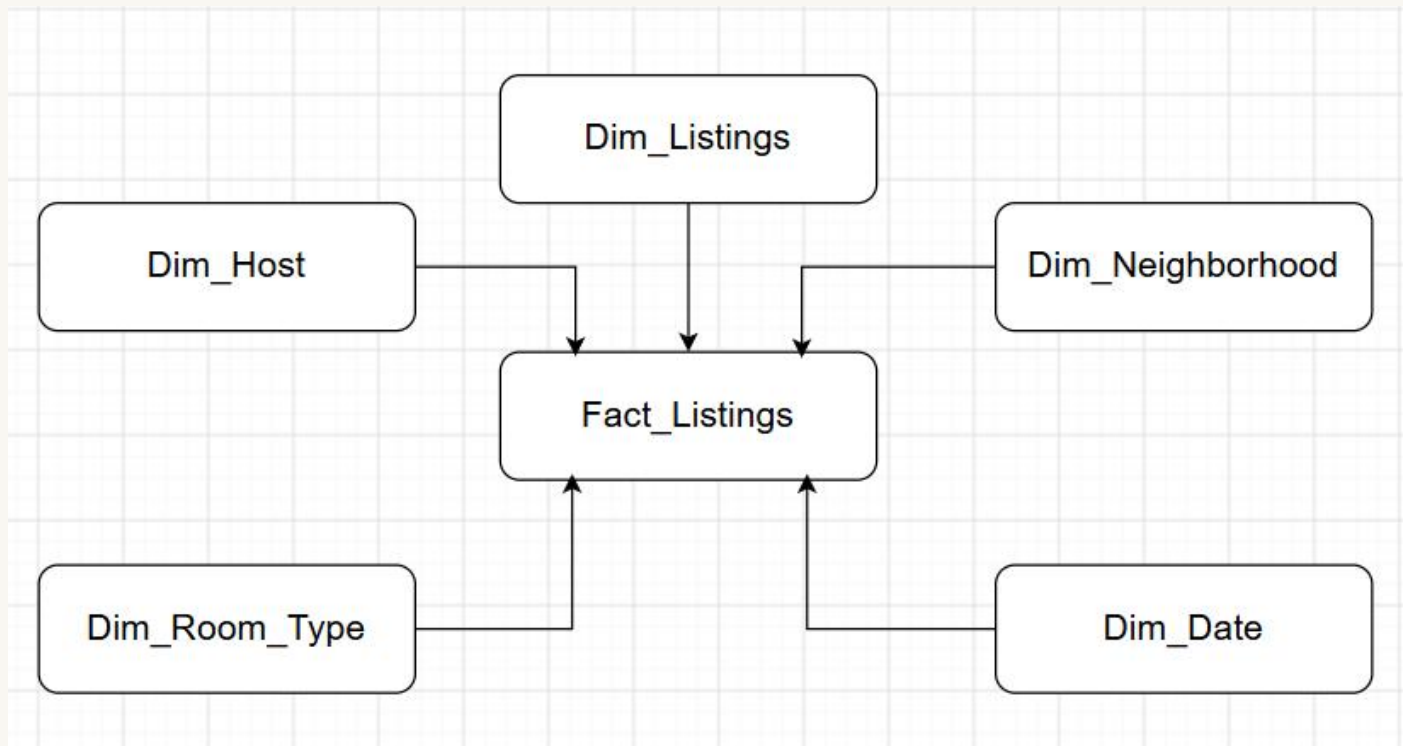
8.1 Model Types Defined

Conceptual Model: A high-level representation of business entities and their relationships, independent of any technology or implementation detail. It defines what the system contains from a business perspective, using plain business terminology understood by stakeholders.

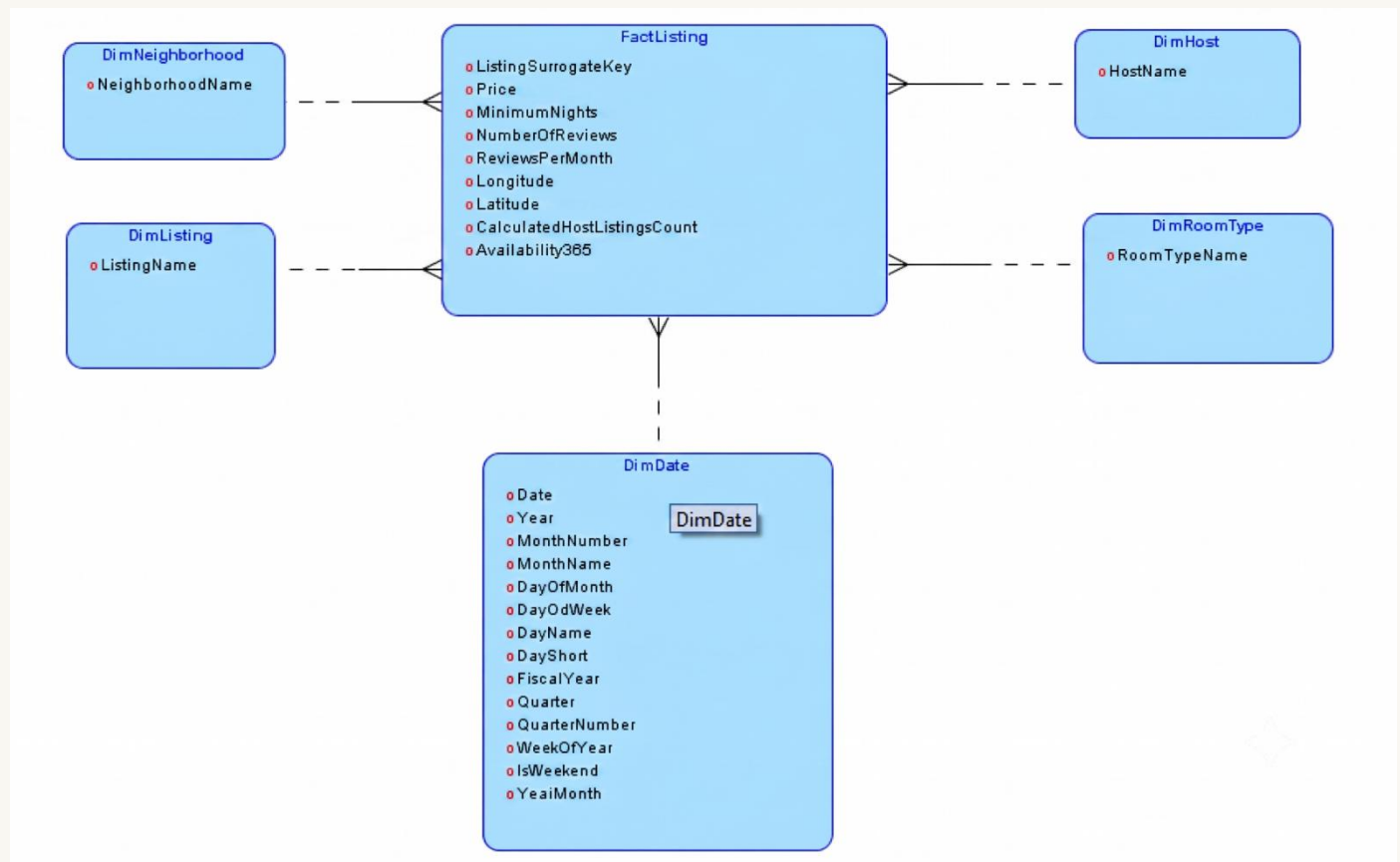
Logical Model: A detailed, technology-agnostic specification of data structures, attributes, data types, keys, and relationships. It defines how data is organized with normalization rules applied, but remains independent of any specific database system.

Physical Model: The implementation-specific schema design tailored to a particular database technology. It defines how data is stored, including table structures, indexes, partitions, data types, and performance optimizations for the target platform.

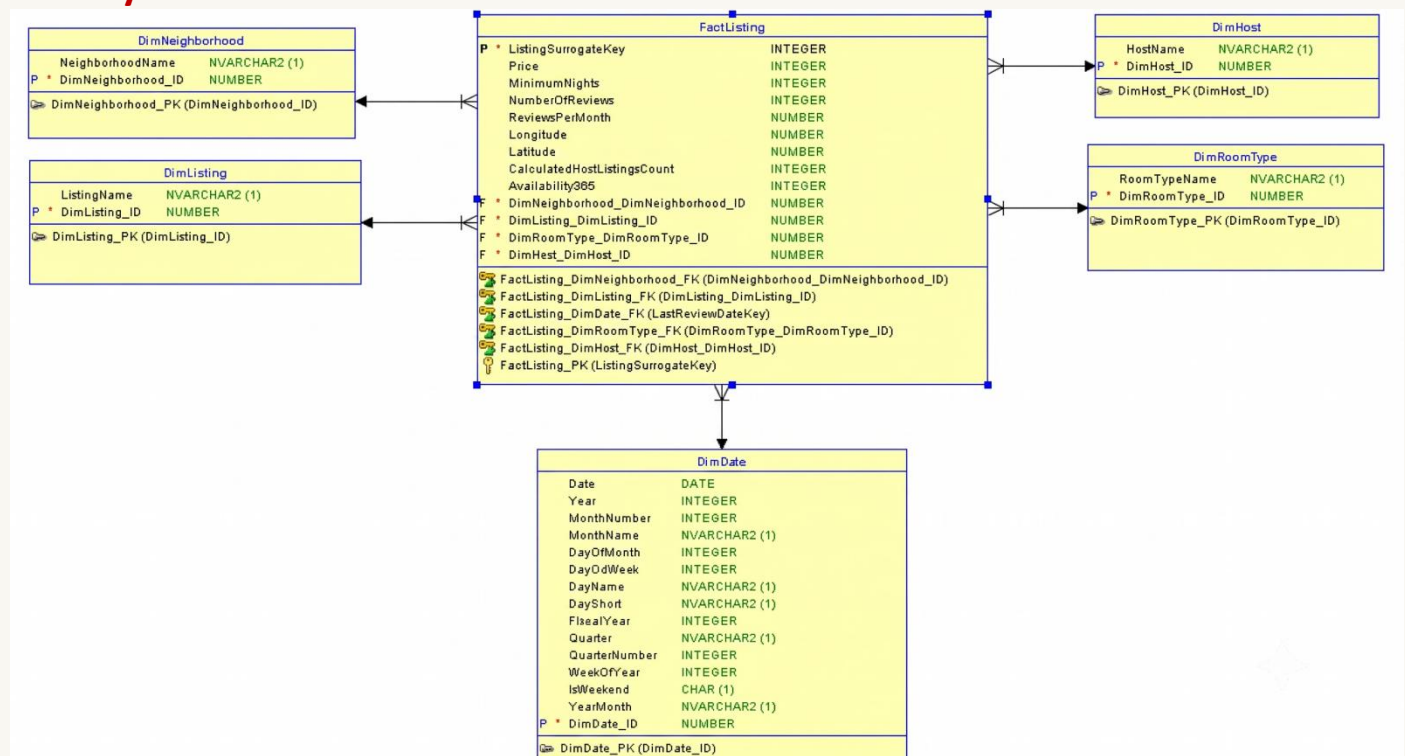
8.2 Conceptual Model



8.3 Logical Model



8.4 Physical Model



Chapter 9: Physical Data Model Implementation

9.1 Overview

The physical data model is implemented in Microsoft Fabric using Dataflow Gen2, transforming cleaned data from the Silver layer into a fully modeled Gold layer star schema. Dataflow Gen2 provides a low-code, Power Query-based ETL experience with visual transformation logic and automated orchestration, enabling efficient data preparation for analytics.

9.2 Architecture

Silver (Cleaned Data) → Dataflow Gen2 → Gold (Modeled Data)

9.3 Dimension Tables

Dimension	Description	Columns	Grain
DimDate	Calendar date dimension with fiscal calendar support	Date, DateKey, Year, MonthNumber, MonthName, MonthShort, DayOfMonth, DayOfWeek, DayName, DayShort, FiscalYear, Quarter, QuarterNumber, WeekOfYear, IsWeekend, YearMonth	Per date
DimHost	Host attributes with calculated categories	HostID (bigint), HostName (varchar), Host Category (varchar)	Per host_id
DimListing	Listing reference details	ListingID (bigint), ListingName (varchar)	Per listing_id
DimNeighborhood	Location hierarchy for NYC boroughs	NeighborhoodID (bigint), NeighborhoodName (varchar),	Per neighborhood

		NeighborhoodGroup (varchar)	
DimRoomType	Room type reference	RoomTypeID (bigint), RoomTypeName (varchar)	Per room_type

DimDate is generated programmatically to cover the full date range present in the dataset, with fiscal year logic (July-June) implemented for business reporting.

9.4 Fact Table

Table	Description	Key Columns	Measures
FactListing	Listing-level fact containing performance metrics and dimension foreign keys	ListingSurrogateKey, ListingID, HostID, NeighborhoodID, RoomTypeID, LastReviewDateKey	Price, MinimumNights, NumberOfReviews, ReviewsPerMonth, CalculatedHostListingsCount, Availability365, Price Tier, Is High Demand, Booking Potential, Price per Month

The fact table includes materialized derived columns for analytical convenience:

- Price Tier: Free / Budget (<\$100) / Mid-Range (<\$300) / Premium (<\$500) / Luxury
- Is High Demand: 1 if NumberOfReviews > 50 AND ReviewsPerMonth > 3
- Booking Potential: Not Available (0 days) / Low (<30) / Medium (<180) / High
- Price per Month: Price × 30 (estimated monthly revenue)

9.5 Data Types Summary

All Gold layer tables use optimized data types for storage and query performance:

Data Type	Usage	Columns
bigint	Surrogate keys, IDs, counts	HostID, ListingID, DateKey, Year, MonthNumber
int	Numeric measures	Price, MinimumNights, NumberOfReviews, Availability365
float	Geospatial coordinates	Latitude, Longitude
decimal(18,2)	Monetary values	Price per Month
varchar	Text attributes	HostName, MonthName,

		Quarter, Price Tier
date	Calendar dates	Date

Important: All `varchar` columns use `varchar` instead of `nvarchar` due to Fabric Warehouse compatibility requirements.

9.6 Dataflow Gen2 Implementation Steps

1. Source Connection: Connect to Silver layer delta tables in the Fabric Lakehouse Files section
2. Dimension Builds:
 - Extract distinct dimension values from fact data
 - Deduplicate and assign surrogate keys
 - Generate `DimDate` for complete date range
3. Fact Build:
 - Join staged data with dimensions to resolve foreign keys
 - Apply derived column calculations (Price Tier, Is High Demand, etc.)
4. Destination: Write output tables to Gold layer Warehouse

9.7 Star Schema Output

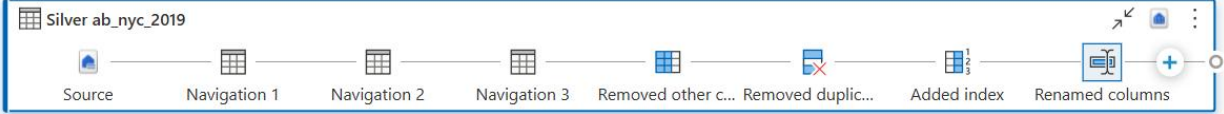
Table	Type	Grain	Surrogate Key
DimDate	Dimension	Per date	DateKey
DimHost	Dimension	Per host_id	HostID
DimListing	Dimension	Per listing_id	ListingID
DimNeighborhood	Dimension	Per neighborhood	NeighborhoodID
DimRoomType	Dimension	Per room_type	RoomTypeID
FactListing	Fact	Per listing	ListingSurrogateKey

9.8 Notes on Implementation

- Surrogate Keys: All dimension keys are natural keys (not generated), maintaining referential integrity with source data
- Derived Columns: Five materialized columns were added to FactListing for analytical convenience, reducing DAX complexity in the semantic layer

9.9 Pictures From DataFlow Gen2

DimNeighborhood

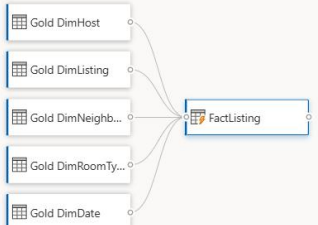


The pipeline for DimNeighborhood starts with a source 'Silver ab_nyc_2019'. It passes through three navigation steps, then a step to remove other columns, followed by removing duplicates, adding an index, and finally renaming columns. The final step is 'Renamed columns'.

Table: `Table.RenameColumns("#Added index", {"neighbourhood", "NeighborhoodName"}, {"neighbourhood_group", "NeighborhoodGroup"}, {"Index", "NeighborhoodID"})`

	NeighborhoodName	NeighborhoodGroup	NeighborhoodID
1	Bedford-Stuyvesant	Brooklyn	1
2	Fort Greene	Brooklyn	2
3	Bushwick	Brooklyn	3
4	Long Island City	Queens	4
5	Brooklyn Heights	Brooklyn	5
6	NoHo	Manhattan	6
7	Two Bridges	Manhattan	7
8	Rockaway Beach	Queens	8
9	Cypress Hills	Brooklyn	9
10	Concord	Staten Island	10
11	Fordham	Bronx	11

FactListing



The pipeline for FactListing starts with five source tables: Gold DimHost, Gold DimListing, Gold DimNeighb..., Gold DimRoomTy..., and Gold DimDate. These are all connected to the FactListing table.

Table: `Table.RenameColumns("#Renamed columns 1", {"DateKey", "LastReviewDateKey"}, {"latitude", "Latitude"}, {"longitude", "Longitude"}, {"price", "Price"}, {"minimum_nights", "MinimumNights"}, {"listing_surrogate_key", "ListingSurrogateKey"}, {"listing_id", "ListingID"}, {"host_id", "HostID"}, {"neighborhood_id", "NeighborhoodID"})`

ListingSurrogateKey	Latitude	Longitude	Price	MinimumNights	NumberOfReviews	ReviewsPerMonth	CalculatedHostListingsCount	Availability365	ListingID	HostID	NeighborhoodID
1	40.64749	-73.97237	149	1	9	0.21	6	365	25501	10143	54
2	40.75362	-73.98377	225	1	45	0.38	2	355	40336	9261	55
3	40.80902	-73.9419	150	3	0	null	1	365	4541	11394	143
4	40.68514	-73.95976	89	1	270	4.64	1	194	26307	2171	84
5	40.79851	-73.94399	80	10	9	0.1	1	0	26727	1526	198
6	40.74767	-73.975	200	3	74	0.59	1	129	26094	3210	209
7	40.68688	-73.95596	60	45	49	0.4	1	0	6787	5705	1
8	40.76489	-73.98493	79	2	430	3.47	1	220	19994	10403	183
9	40.80178	-73.96723	79	2	118	0.99	1	0	10700	9341	15
10	40.71344	-73.99037	150	1	160	1.33	4	188	18765	1369	85

Chapter 10: Semantic Layer

10.1 Overview

The semantic layer connects the Silver dimensional model to end-user reporting by defining KPIs, calculated columns, and business logic. Due to Fabric privilege restrictions preventing calculated column creation in the data model, SQL scripts were used to materialize derived columns directly in the Gold layer tables, while DAX measures were defined in the Power BI model for dynamic aggregations.

10.2 Architecture

Silver Layer → Gold Layer (Materialized Columns via SQL) → Power BI Model (DAX Measures) → Reports

10.3 Core KPIs

#	Measure	DAX Formula	Description
1	Total Listings	COUNTROWS('Gold FactListing')	Total number of listings
2	Avg Price	AVERAGE('Gold FactListing'[Price])	Mean listing price
3	Median Price	MEDIAN('Gold FactListing'[Price])	Median listing price
4	Total Hosts	DISTINCTCOUNT('Gold FactListing'[HostID])	Unique host count
5	Avg Reviews	AVERAGE('Gold FactListing'[NumberOfReviews])	Mean reviews per listing
6	Avg Availability	AVERAGE('Gold FactListing'[Availability365])	Mean available days
7	Total Reviews	SUM('Gold FactListing'[NumberOfReviews])	Sum of all reviews
8	Occupancy Rate	DIVIDE([Avg Availability], 365, 0)	Ratio of available days to year

10.4 Price Analysis

#	Measure	DAX Formula
9	Min Price	MIN('Gold FactListing'[Price])
10	Max Price	MAX('Gold FactListing'[Price])
11	Price Range	[Max Price] - [Min Price]
12	Listings Under 100	CALCULATE([Total Listings], 'Gold FactListing'[Price] < 100)
13	Listings 100-300	CALCULATE([Total Listings], 'Gold FactListing'[Price] >= 100 && 'Gold FactListing'[Price] < 300)
14	Listings Over 300	CALCULATE([Total Listings], 'Gold FactListing'[Price] >= 300)
15	Pct Under 100	DIVIDE([Listings Under 100], [Total Listings], 0)

10.5 Room Type Analysis

#	Measure	DAX Formula
16	Entire Home Count	CALCULATE([Total Listings], 'Gold DimRoomType'[RoomTypeName] = "Entire home/apt")
17	Private Room Count	CALCULATE([Total Listings], 'Gold DimRoomType'[RoomTypeName] = "Private room")
18	Shared Room Count	CALCULATE([Total Listings], 'Gold DimRoomType'[RoomTypeName] = "Shared room")
19	Avg Price by Room	AVERAGE('Gold FactListing'[Price])

10.6 Neighborhood & Borough Analysis

#	Measure	DAX Formula	Usage
20	Listings by Borough	[Total Listings]	Use with NeighborhoodGroup
21	Avg Price by Borough	[Avg Price]	Use with NeighborhoodGroup
22	Listings Count	[Total Listings]	Visual filter, Top N = 10

10.7 Host Analysis

#	Measure	DAX Formula
23	Super Host Threshold	10 (constant)
24	Avg Listings per Host	DIVIDE([Total Listings], [Total Hosts], 0)
25	Single Listing Hosts	COUNTROWS(FILTER(SUMMARIZE('Gold FactListing', 'Gold FactListing'[HostID], "Count", COUNTROWS('Gold FactListing')), [Count] = 1))
26	Multi Listing Hosts	[Total Hosts] - [Single Listing Hosts]

10.8 Review Analysis

#	Measure	DAX Formula
27	Listings with Reviews	CALCULATE([Total Listings], 'Gold FactListing'[NumberOfReviews] > 0)
28	Listings without Reviews	CALCULATE([Total Listings], 'Gold FactListing'[NumberOfReviews] = 0)
29	Pct with Reviews	DIVIDE([Listings with Reviews], [Total Listings], 0)
30	Avg Reviews per Month	AVERAGE('Gold FactListing'[ReviewsPerMonth])

10.9 Time-Based Measures

#	Measure	DAX Formula	Usage
35	Reviews by Year	SUM('Gold FactListing'[NumberOfReviews])	Use with DimDate[Year]
36	Listings by Review Year	[Total Listings]	Use with DimDate[Year]

10.10 Materialized Columns (SQL Implementation)

Due to Fabric privilege restrictions, calculated columns were materialized directly in Gold layer tables using SQL scripts instead of DAX calculated columns.

Gold.FactListing Columns Added:

Column	Logic
Price Tier	Free / Budget (<\$100) / Mid-Range (<\$300) / Premium (<\$500) / Luxury
Is High Demand	1 if NumberOfReviews > 50 AND ReviewsPerMonth > 3
Booking Potential	Not Available (0 days) / Low (<30) / Medium (<180) / High
Price per Month	Price × 30

Gold.DimHost Column Added:

Column	Logic
Host Category	Single Listing (1) / Small Host (≤5) / Medium Host (≤20) / Large Host

10.11 SQL Implementation Script

```
'''
```

```
-- Add columns
```

```
ALTER TABLE Gold.FactListing
```

```
ADD [Price Tier] VARCHAR(50), [Is High Demand] INT,  
    [Booking Potential] VARCHAR(50), [Price per Month] DECIMAL(18,2);
```

```
ALTER TABLE Gold.DimHost
```

```
ADD [Host Category] VARCHAR(50);
```

```

-- Populate FactListing
UPDATE Gold.FactListing
SET [Price Tier] = CASE
    WHEN Price = 0 THEN 'Free' WHEN Price < 100 THEN 'Budget'
    WHEN Price < 300 THEN 'Mid-Range' WHEN Price < 500 THEN 'Premium'
    ELSE 'Luxury' END,
[Is High Demand] = CASE WHEN NumberOfReviews > 50 AND ReviewsPerMonth > 3 THEN 1 ELSE 0 END,
[Booking Potential] = CASE
    WHEN Availability365 = 0 THEN 'Not Available' WHEN Availability365 < 30 THEN 'Low'
    WHEN Availability365 < 180 THEN 'Medium' ELSE 'High' END,
[Price per Month] = Price * 30;

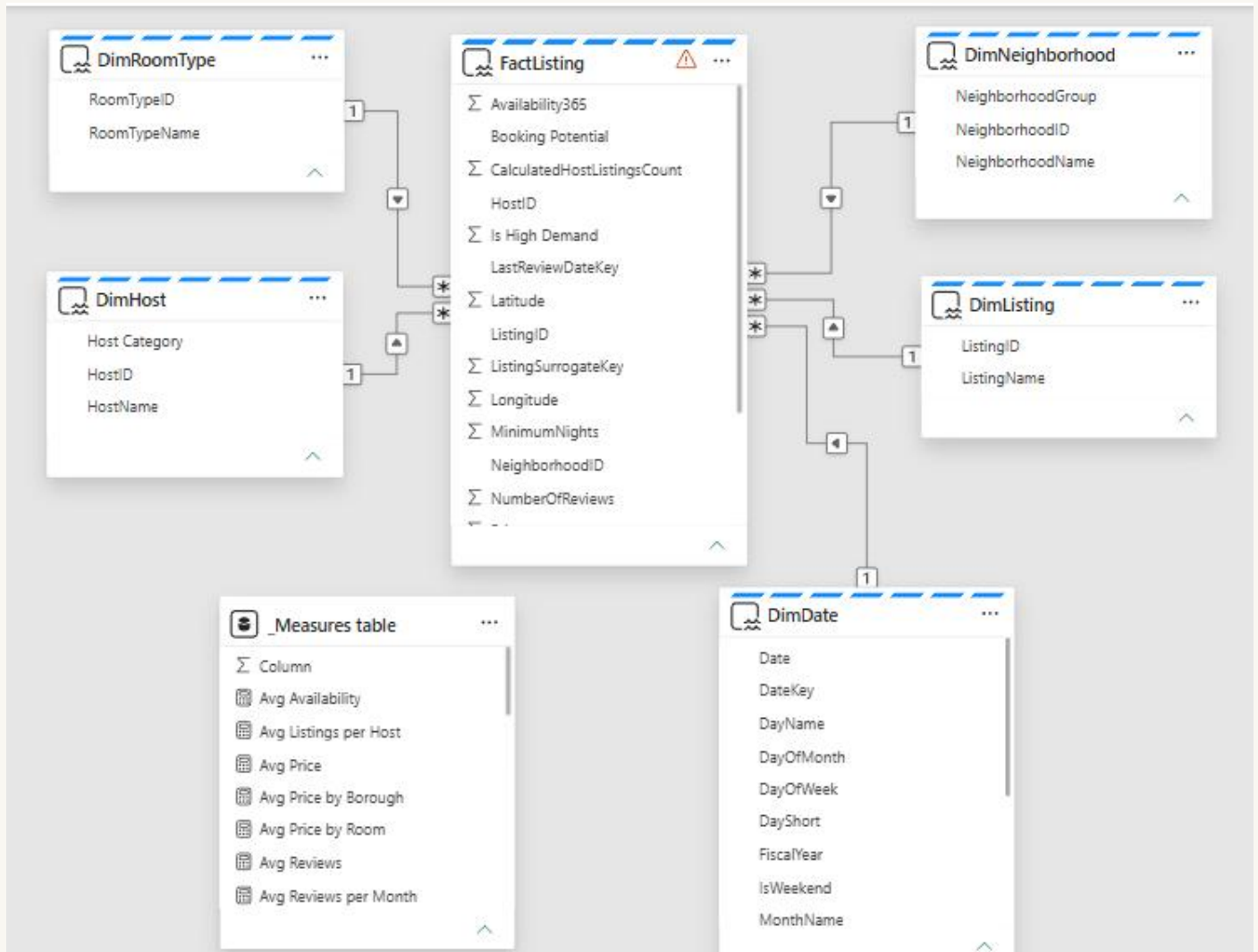
-- Populate DimHost
WITH HostListingCounts AS (
    SELECT HostID, COUNT(*) AS ListingCount FROM Gold.FactListing GROUP BY HostID
)
UPDATE Gold.DimHost
SET [Host Category] = CASE
    WHEN hlc.ListingCount = 1 THEN 'Single Listing'
    WHEN hlc.ListingCount <= 5 THEN 'Small Host'
    WHEN hlc.ListingCount <= 20 THEN 'Medium Host'
    ELSE 'Large Host' END
FROM Gold.DimHost dh
INNER JOIN HostListingCounts hlc ON dh.HostID = hlc.HostID;
'''

```

10.12 Measure Summary

Category	Measures
Core KPIs	8
Price Analysis	7
Room Type	4
Borough	3
Host	4
Reviews	4
Time-Based	2
Total Measures	32
Materialized Columns	5

10.13 Picture of the Semantic Model



Chapter 11: SMART Questions

Smart Question 1: What is the current state of the NYC Airbnb market in 2019?

Specific: Quantify the total listings, hosts, average price, and availability across all NYC boroughs.

Measurable: Tracked via 5 core KPIs — Total Listings, Average Price, Total Hosts, Average Reviews, Occupancy Rate.

Achievable: Data is complete for all 48,895 listings across 5 boroughs with 16 attributes per listing.

Relevant: Provides macro-level market snapshot for executive stakeholders and policy makers.

Time-bound: Represents 2019 calendar year snapshot with review activity spanning 2011-2019.

Smart Question 2: How does pricing vary by room type and neighborhood, and where are the outliers?

Specific: Analyze price distribution across room types and neighborhoods to identify premium areas and pricing anomalies.

Measurable: Avg Price by Room Type, Avg Price by Borough, Price Tier Distribution, Top 10 Most Expensive Neighborhoods, Price Outlier Count.

Achievable: 674 distinct price points across 3 room types and 221 neighborhoods.

Relevant: Enables market analysts to identify pricing strategy opportunities and anomalous listings.

Time-bound: 2019 pricing snapshot with 388 statistical outliers identified (3σ above \$873 mean).

Smart Question 3: Which neighborhoods offer the best balance of availability, price, and reviews?

Specific: Identify neighborhoods that optimize the trade-off between affordability, bookability, and demand signals.

Measurable: Listings Count by Neighborhood, Avg Price, Avg Availability, Avg Reviews, Booking Potential classification.

Achievable: 221 neighborhoods with complete pricing, availability, and review metrics.

Relevant: Supports real estate strategists and travelers in identifying high-value market segments.

Time-bound: 2019 data with availability ranging 0-365 days and reviews spanning 2011-2019.

Smart Question 4: Who are the power hosts, and how is the market split between individuals and commercial operators?

Specific: Identify multi-listing hosts, categorize host types, and measure market concentration.

Measurable: Total Hosts, Single Listing Hosts, Multi Listing Hosts, Avg Listings per Host, Host Category Distribution, Top 10 Hosts by Listing Count.

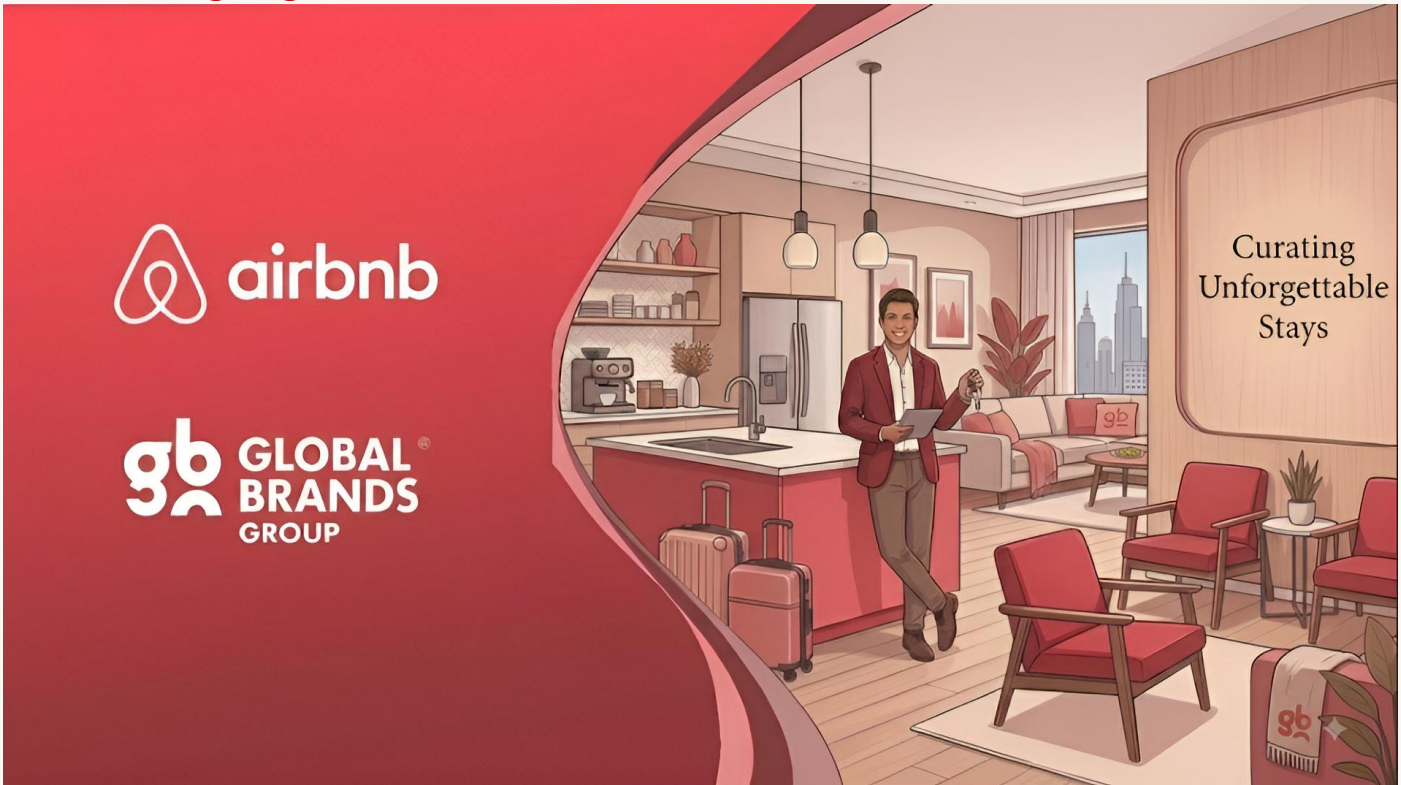
Achievable: 37,457 unique hosts with listing counts ranging from 1 to 327.

Relevant: Reveals market concentration risk and the balance between casual individuals and professional operators.

Time-bound: 2019 host snapshot with calculated host listing counts validated against actual row counts.

Chapter 12: Visualization Using PowerBI

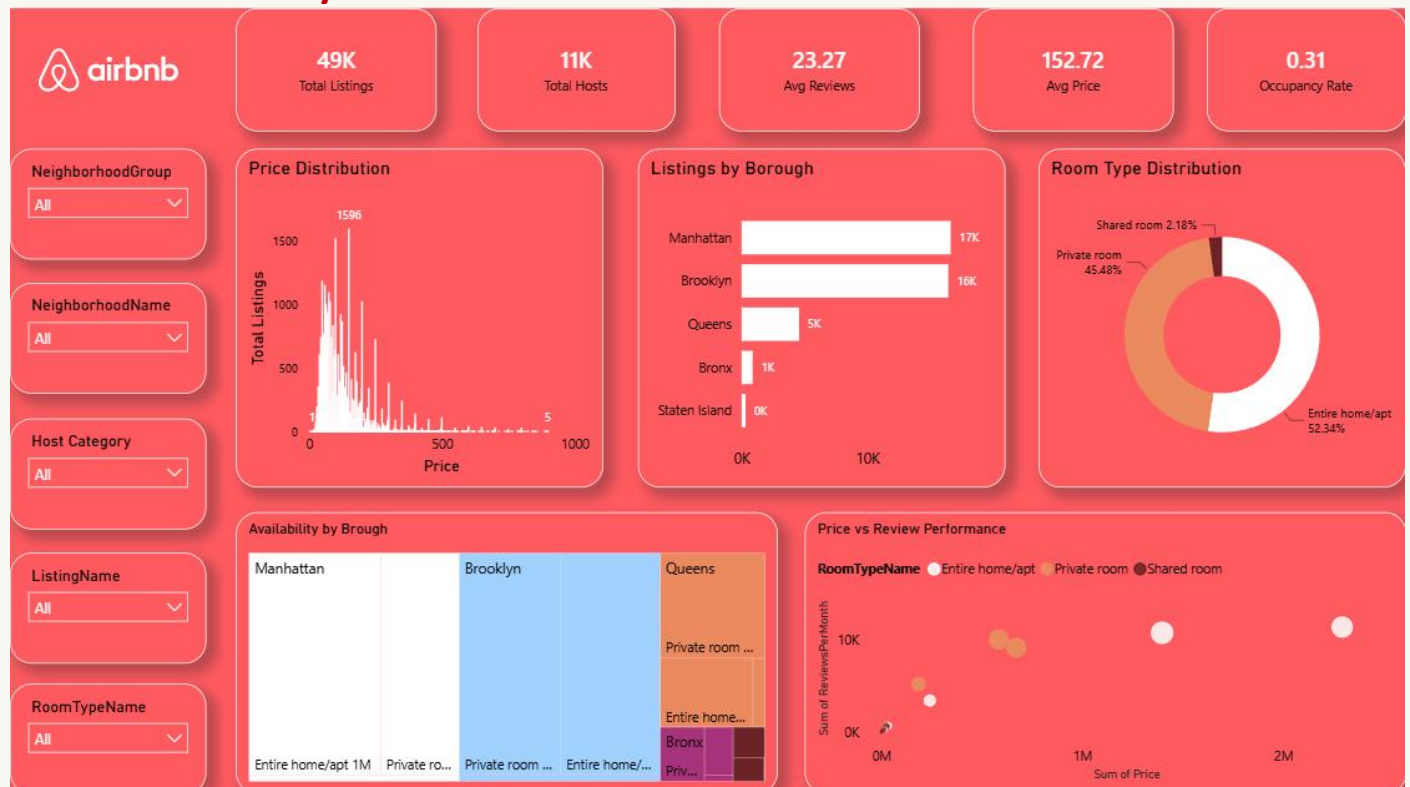
12.0 Landing Page



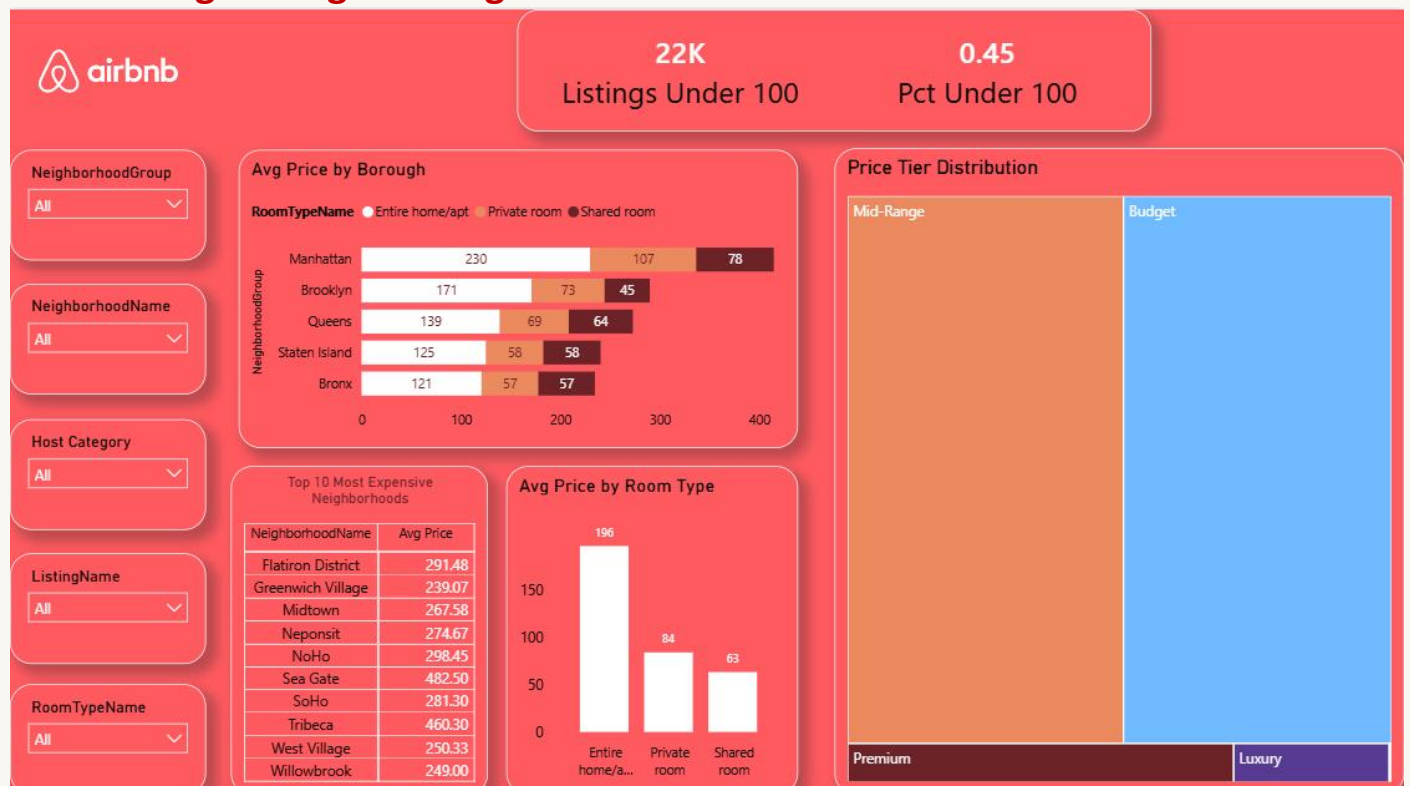
12.1 Overview Page



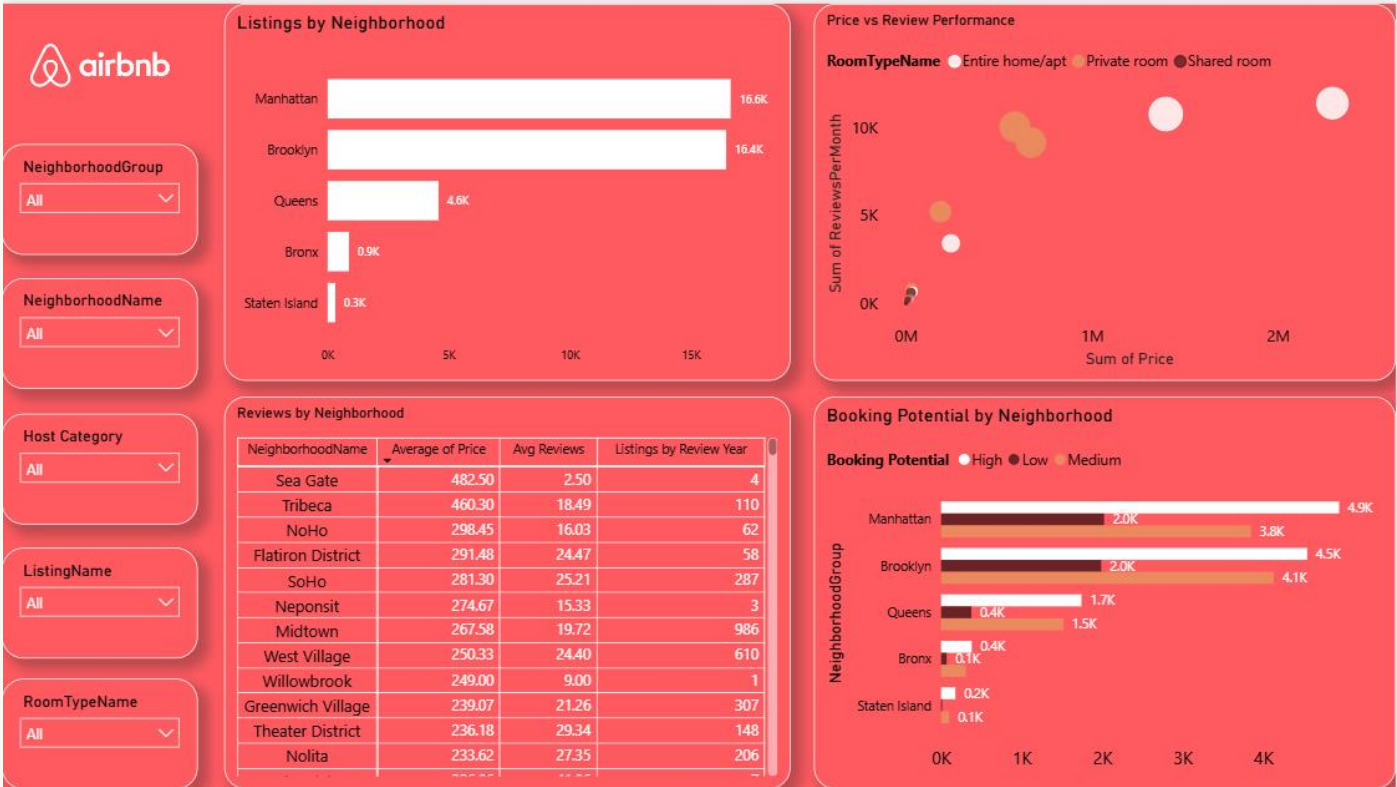
12.2 Market Analysis



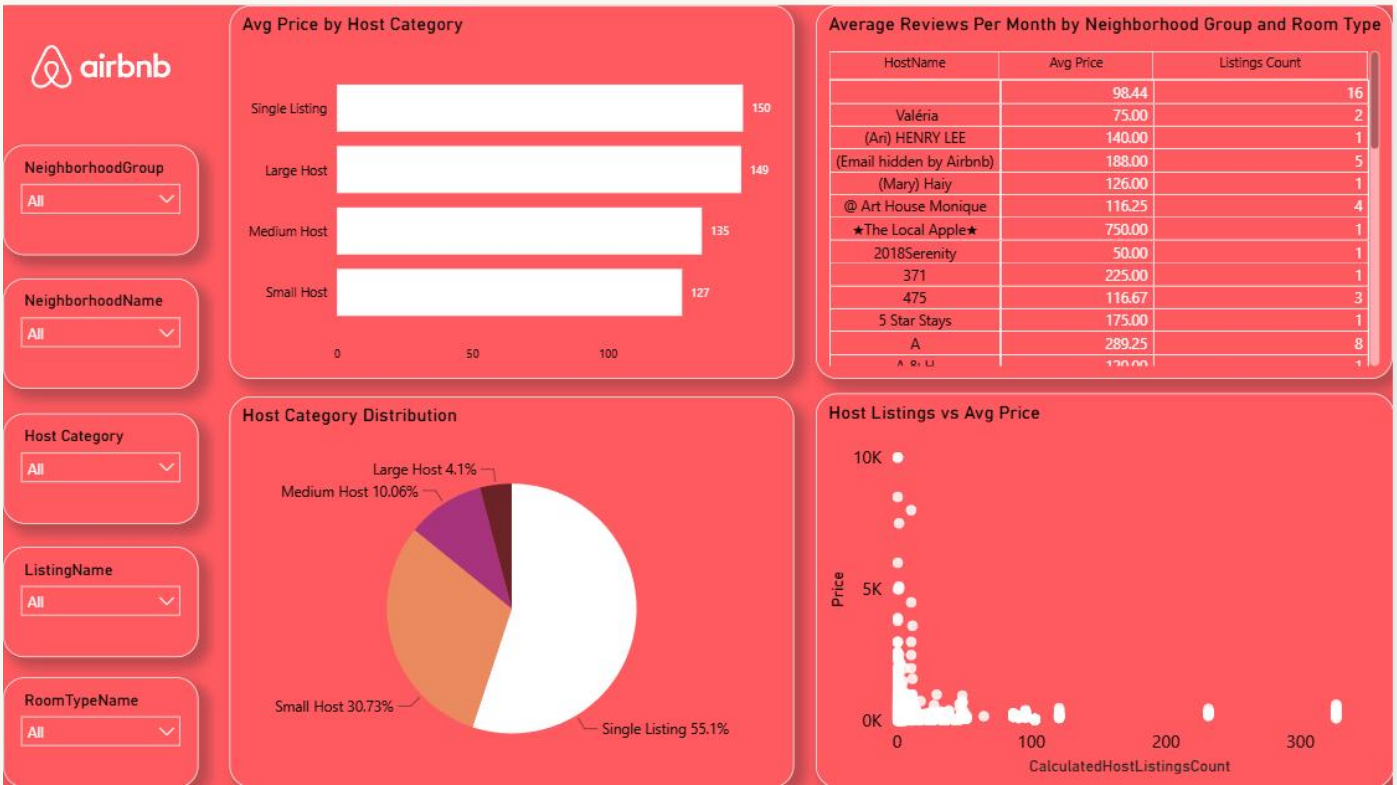
12.3 Pricing Intelligence Page



12.4 Neighborhood Deep Dive Page



12.5 Host Analysis Page



THANK YOU